

Automata and Formal Languages

Dr. Sankhadeep Chatterjee

Properties of Regular Languages

- Which languages are not regular?
- Decision properties
 - A decision property for a class of languages is an algorithm that takes a formal description of a language (e.g., a DFA) and tells whether or not some property holds.
 - Is a regular language empty?
 - Is a string w belong to the language?
 - Do different descriptions of languages describe same language?

Closure properties

- A *closure property* of a language class says that given languages in the class, an operator (e.g., union) produces another language in the same class.
 - The union of two regular languages is regular
 - The intersection of two regular languages is regular
 - The complement of a regular language is regular
 - The difference of two regular languages is regular
 - The reversal of a regular language is regular
 - The closure (star) of a regular language is regular
 - The concatenation of regular languages is regular
 - A homomorphism (substitution of strings for symbols) of a regular language is regular
 - The inverse homomorphism of a regular language is regular

Pumping Lemma for Regular Languages

For every regular language L Number of states of
DFA for L

There is an integer n , such that

For every string w in L of length $\geq n$

We can write $w = xyz$ such that:

1. $|xy| \leq n$.
2. $|y| > 0$.
3. For all $k \geq 0$, xy^kz is in L .

Proof: Pumping Lemma for Regular Languages

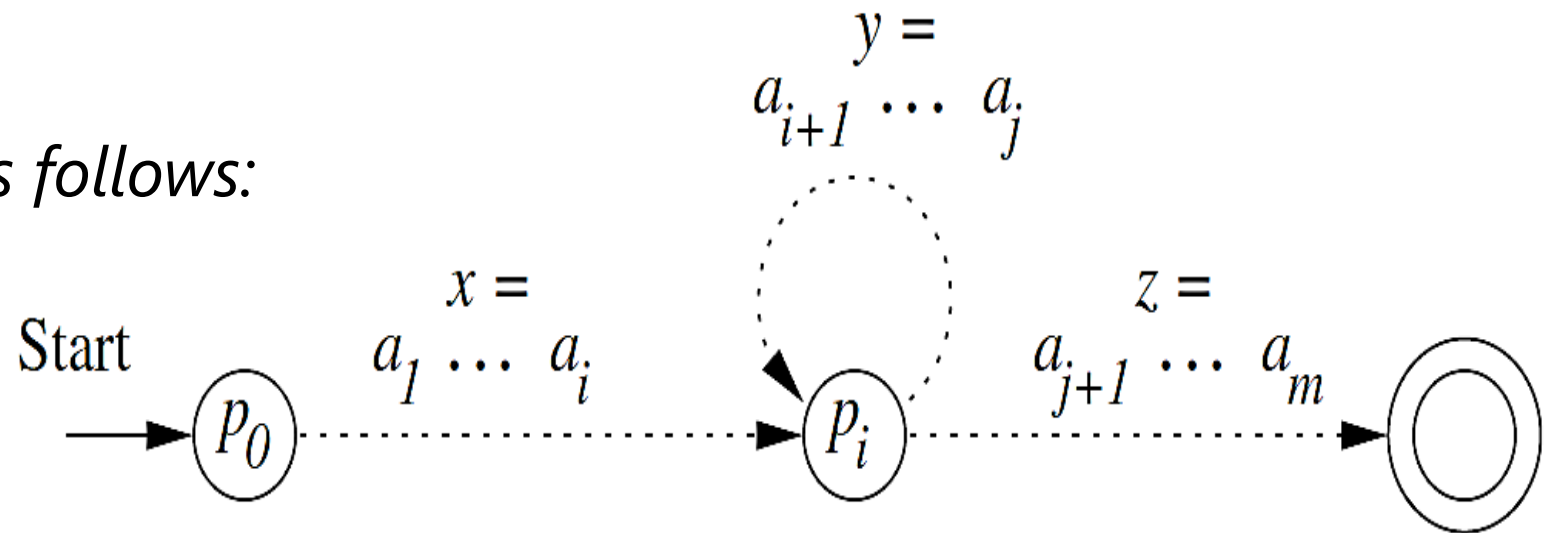
- Suppose L is regular. Then $L = L(A)$ for some DFA A with n states.
- Now, consider any string w of length n or more,
say $w = a_1 a_2 \dots a_m$, where $m \geq n$ and each a_i is an input symbol
- For $i = 0, 1, \dots, n$ define state p_i to be $\hat{\delta}(p_0, a_1 a_2 \dots a_i)$,
where $\hat{\delta}$ is the extended transition function of A , and p_0 is the start state of A .
- That is, p_i is the state where A is in after reading the first i symbols of w .

Proof: Pumping Lemma for Regular Languages

- *By the pigeonhole principle, it is not possible for the $n + 1$ different p_i 's for $i = 0, 1, \dots, n$ to be distinct, since there are only n different states.*
- Thus, we can find two different integers i and j , with $0 \leq i < j \leq n$, such that $p_i = p_j$.

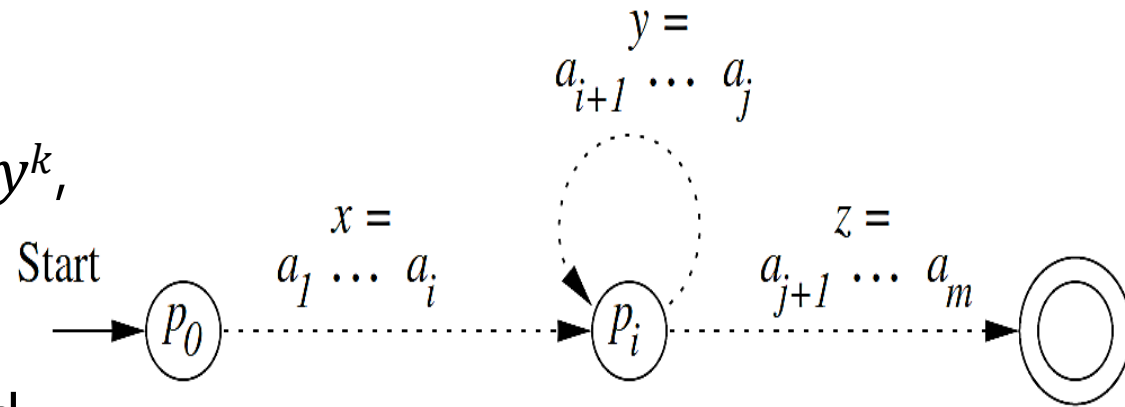
- *We can break $w = xyz$ as follows:*

- $x = a_1 a_2 \dots a_i$
- $y = a_{i+1} a_{i+2} \dots a_j$
- $z = a_{j+1} a_{j+2} \dots a_m$



Proof: Pumping Lemma for Regular Languages

- Now, consider input xy^kz for any $k \geq 0$.
- If $k = 0$,
 - The automaton goes from the start state p_0 to p_i on input x .
 - Since p_i is also p_j , it must be that A goes from p_i to the accepting state on input z . Thus, A accepts xz .
- If $k > 0$,
 - then A goes from p_0 to p_i on input x ,
 - circles from p_i to p_j , k times on input y^k ,
 - then goes to the accepting state on input z .
- Thus, for any $k > 0$, xy^kz is also accepted by A ; that is, xy^kz is in L .



Pumping Lemma contd.

- Finite Languages: If n is chosen as one more than the maximum length of any string in L , the lemma is vacuously true because no string has length $\geq n$.
- Therefore, the Pumping Lemma holds for all regular languages.
- For all regular languages: $R(L) \Rightarrow P(L)$, where $P(L)$ is the property defined by the lemma.

Pumping Lemma contd.

- We use pumping lemma to show that a given language is not regular.
- To prove L is not regular, we show it does not satisfy the Pumping Lemma ($\neg P(L) \Rightarrow \neg R(L)$).
- The Negation Logic:
 - For any $n \geq 0$.
 - There exists a string $w \in L$ with $|w| \geq n$.
 - For all partitions xyz where $y \neq \epsilon$.
 - There exists an $i \geq 0$ such that $xy^iz \notin L$
- If the conditions stated in pumping lemma is satisfied it can not be concluded that the language is regular or not ($P(L) \not\Rightarrow R(L)$).

The Adversarial Game

- The proof can be viewed as a game against an opponent:
 - Opponent: Gives you an arbitrary number n .
 - You: Smartly pick a string $w \in L$ where $|w| \geq n$.
 - Opponent: Divides w into xyz with $y \neq \epsilon$ in any way they wish.
 - You: Identify an i such that xy^iz is not in L .

Example: Pumping Lemma

- Show that $L = \{w \mid w \text{ is palindrome}\}$ over $\Sigma = \{a, b\}$ is not Regular
- Suppose, L is regular. Then there exists a constant 'n' such that it satisfies all conditions of Pumping Lemma.
- Let $w = a^nba^n$ (Note: $|w| \geq n$)
- Using pumping lemma w is broken as xyz such $y \neq \epsilon$ and $|xy| \leq n$
- Hence, both x and y contains only a 's
- Assume that, $x = a^i$ and $y = a^j$, thus $z = ba^n$
- Now, by pumping lemma, $xy^0z = a^i\epsilon ba^n = a^{n-j}ba^n \in L$
- However, $a^{n-j}ba^n \notin L$
- Hence, our assumption was incorrect. L is not regular

Example: Pumping Lemma

- Show that $L = \{1^p \mid p \text{ is a prime}\}$ not Regular.
- Consider, L is regular, $w = 1^p \in L$ and $p \geq n + 2$
- Using pumping lemma w is broken as xyz such $y \neq \epsilon$ and $|xy| \leq n$
- Assume that $|y| = m$ and $|xz| = p - m$
- Consider the string $xy^{p-m}z$
- $|xy^{p-m}z| = |xz| + |y|(p - m) = p - m + m(p - m) = (m + 1)(p - m)$
- As, $y \neq \epsilon$, $m \geq 1$. Hence, $m + 1 > 1$
- As, $p \geq n + 2$ and $m \leq n$, $p - m > 1$
- Hence, $|xy^{p-m}z|$ is not prime and $|xy^{p-m}z| \notin L$
- Thus, L is not regular.

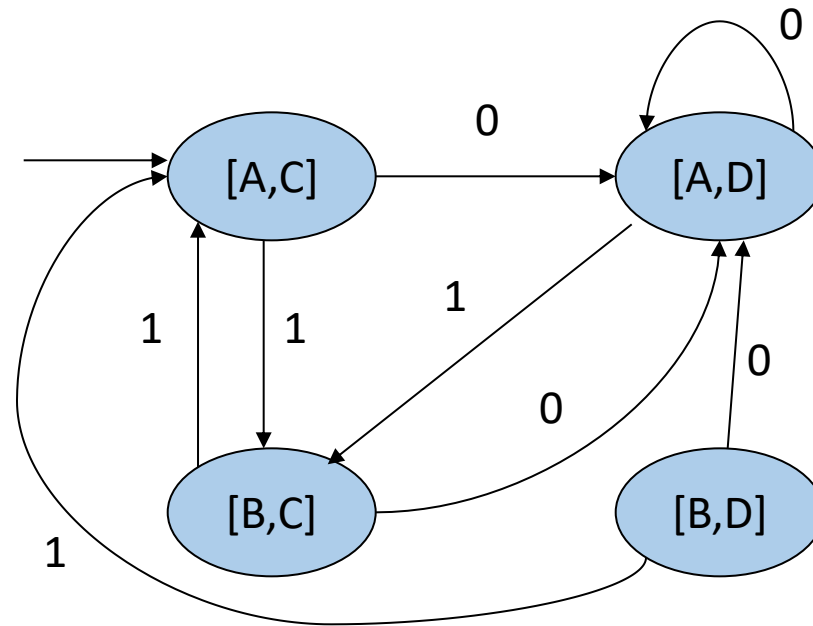
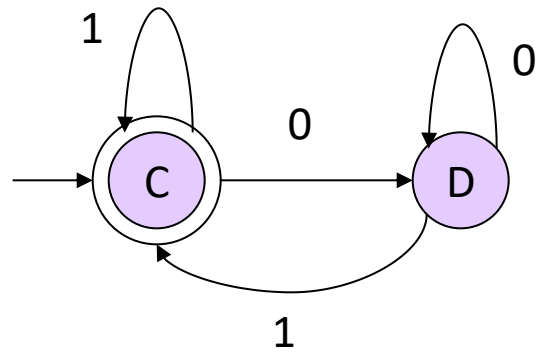
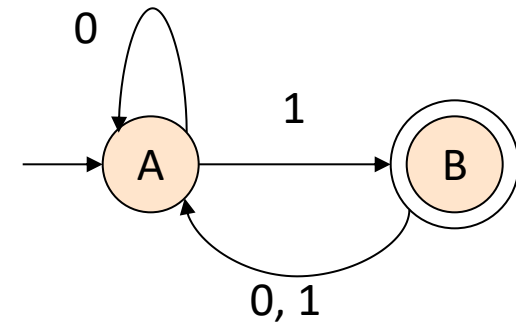
Is a regular language empty?

- Given a regular language, does the language contain any string at all.
- Assume representation is DFA.
- Construct the transition graph.
- Compute the set of states reachable from the start state.
- If any final state is reachable, then yes, else no.

Given regular languages L and M, is $L = M$?

- Construct the **Product DFA** from DFA's for L and M.
- Let these DFA's have sets of states Q and R, respectively.
- Product DFA has set of states $Q \times R$.
 - i.e., pairs $[q, r]$ with q in Q, r in R.
- Start state = $[q_0, r_0]$ (the start states of the DFA's for L, M).
- Transitions: $\delta([q, r], a) = [\delta_L(q, a), \delta_M(r, a)]$
 - δ_L, δ_M are the transition functions for the DFA's of L, M.
 - That is, we simulate the two DFA's in the two state components of the product DFA.

Example: Product DFA



Given regular languages L and M , is $L = M$?

- Make the final states of the product DFA be those states $[q, r]$ such that exactly one of q and r is a final state of its own DFA.
- Thus, the product accepts w iff w is in exactly one of L and M .
- The product DFA's language is empty iff $L = M$.
- Use Emptiness checking algorithm to test whether the language of a DFA is empty.

Do they accept same language?

